

Glossary, Definitions & Pro Tips
Companion to the Platform Engineering Handbook

Gopal Khandale

2026

Contents

How to use this companion	4
Section A — Cloud Fundamentals	5
Cloud	5
IaaS, PaaS, SaaS	5
Region	6
Availability Zone (AZ)	6
Resource Group	6
Section B — Linux & Shell	8
Shell	8
Pipe ()	8
Background process (&)	8
SSH	9
SSH LocalForward (port forwarding)	9
systemd	9
Section C — Git	11
Repository (repo)	11
Commit	11
Branch	11
Pull Request (PR) / Merge Request (MR)	12
Rebase vs Merge	12
Merge conflict	12
Section D — Networking	14
IP Address	14
CIDR (Classless Inter-Domain Routing)	14
Subnet	14
NSG (Network Security Group)	15
DNS	15
Port	15
TLS / HTTPS	15
mTLS (mutual TLS)	16
Load Balancer	16
Section E — Containers	17

Container	17
Image	17
Dockerfile	17
Layer	18
Multi-stage build	18
Registry	18
Image digest	18
Section F — Kubernetes	20
Pod	20
Node	20
Namespace	20
Deployment	21
ReplicaSet	21
StatefulSet	21
Service	21
ConfigMap	22
Secret	22
PersistentVolume / PersistentVolumeClaim	22
Ingress / Gateway	22
ServiceAccount	23
RBAC (Role-Based Access Control)	23
Controller	23
CRD (Custom Resource Definition)	24
Operator	24
Section G — Terraform	25
Infrastructure as Code (IaC)	25
State (Terraform state)	25
Remote backend	25
Provider	26
Module	26
State locking	26
Plan vs Apply	26
Drift	27
Section H — Helm	28
Helm	28
Chart	28
Release	28
Values	28
Umbrella chart	29
Section I — Azure	30
Service Principal (SP)	30
Managed Identity	30
Workload Identity (AKS)	30
AKS (Azure Kubernetes Service)	31

ACR (Azure Container Registry)	31
Log Analytics Workspace (LAW)	31
Section J — CI/CD	32
CI (Continuous Integration)	32
CD (Continuous Delivery vs Deployment)	32
Pipeline	32
Artifact	32
GitOps	33
Image promotion (build-once-deploy-many)	33
Section K — Progressive Delivery	34
Blue/Green deployment	34
Canary deployment	34
Argo Rollouts	34
AnalysisTemplate	35
Min-traffic gate	35
Section L — Observability	36
Metric	36
Cardinality	36
PromQL	36
SLI / SLO / SLA	36
Error budget	37
Reading the Handbook — Pro Tips	38
Common Cross-Cutting Tips	39
When you hit an error	39
When learning a new tool	39
When debugging Kubernetes	39
When writing code/YAML/HCL	39
Career tips	40
End of Glossary	41

How to use this companion

Keep this open alongside the main handbook. When you hit a term in the handbook that's unfamiliar, look it up here.

Each entry has:

- **Term** — the word/phrase
 - **Plain English** — what it actually means
 - **Technical definition** — the formal one
 - **Why it matters** — when you'd actually use it
 - **Gotcha** — the thing that trips people up
 - **Interview line** — how to drop it in a conversation
-

Section A — Cloud Fundamentals

Cloud

Plain English	Renting computers and services from a big provider like Microsoft, Amazon, or Google instead of buying them yourself.
Technical	Pay-per-use access to compute, storage, networking, and services exposed via APIs, hosted in geographically distributed data centers.
Why it matters	You can scale from one server to thousands in minutes, pay only for what you use, and rebuild your entire infrastructure as code.
Gotcha	Cloud costs spiral when nobody's watching. A forgotten VM is \$80/month. Always tag resources and set budgets.
Interview line	"The cloud lets us treat infrastructure like cattle, not pets — destroyable and re-creatable from code."

IaaS, PaaS, SaaS

Plain English	How much of the stack the provider runs versus you.
Technical	IaaS = you run OS up. PaaS = you run app up. SaaS = you run nothing.
Why it matters	Picking the right level avoids over-engineering. Don't run a VM if a PaaS will do.
Gotcha	Lock-in increases as you go up the stack. SaaS is easy to start, painful to leave.
Interview line	"We picked AKS (managed K8s, a PaaS) instead of self-managed K8s on VMs because we want the cloud handling control plane and certificate rotation, not us."

Region

Plain English	A geographic area with cloud data centers.
Technical	A set of data centers within the same physical location, low-latency networked, usually 3 AZs.
Why it matters	Latency to users, data residency (GDPR), service availability (some services aren't in all regions), disaster recovery.
Gotcha	Costs vary by region. <code>eastus</code> is usually cheapest, <code>centralindia</code> is sometimes 20% more.
Interview line	"We picked <code>eastus2</code> because it has the latest SKUs and broad service coverage. For HA we'd pair it with <code>westus2</code> ."

Availability Zone (AZ)

Plain English	One independent data center within a region.
Technical	A physically isolated location with independent power, cooling, and network. Tied to a region.
Why it matters	If a data center burns down, your workloads in other AZs keep running.
Gotcha	AZ-aware scheduling costs more (cross-AZ network charges). Some workloads benefit, others don't.
Interview line	"We spread our AKS node pool across 3 AZs so a data center failure doesn't take us out. Cross-AZ pod traffic costs slightly more but the reliability gain pays for it."

Resource Group

Plain English	A folder for related Azure resources.
Technical	A logical Azure container whose lifecycle is managed together. RBAC and tags can apply at this level.
Why it matters	<code>az group delete</code> cleans up everything inside in one call. Simplifies cost allocation.
Gotcha	A resource can only be in ONE RG. Moving resources between RGs is supported but tricky.
Interview line	"We use one RG per environment-app combination — <code>gskplat-rg-platform</code> for the cluster. Makes teardown trivial and cost reporting clean."

Section B — Linux & Shell

Shell

Plain English	The thing that reads what you type and runs it.
Technical	A command interpreter that provides a user interface to the OS. Common ones: bash, zsh, fish.
Why it matters	You'll spend hours every day in a shell.
Gotcha	Mastering it 5x's your productivity. bash and zsh have subtle differences — <code>[\$x -eq 5]</code> works in bash, <code>[[\$x -eq 5]]</code> is the modern way.
Tip	Learn one shell deeply (bash for portability, zsh for daily use). Memorize 10 patterns, copy-paste the rest.

Pipe (|)

Plain English	Takes the output of one command and feeds it to another.
Technical	A Unix construct that connects stdout of one process to stdin of another via a kernel pipe.
Why it matters	Lets you chain simple commands into powerful workflows.
Example	<code>ps aux grep node awk '{print \$2}'</code> → list processes, filter for “node”, extract PIDs.
Gotcha	Pipes are stream-oriented. They start everything in parallel — the second command starts before the first finishes.

Background process (&)

Plain English	Run something but don't wait for it.
----------------------	--------------------------------------

Technical	Forks the process so the shell returns control immediately. The process keeps running.
Example	<code>kubect1 port-forward svc/web 8080:80 &</code>
Gotcha	Closing the terminal usually kills background jobs. Use <code>nohup</code> , <code>disown</code> , or <code>screen/tmux</code> to keep them.

SSH

Plain English	A secure way to log into a remote computer.
Technical	Secure Shell — an encrypted protocol for remote access, file transfer (<code>scp/sftp</code>), and tunneling.
Why it matters	Every server you work with is reached via SSH.
Gotcha	Use SSH keys, not passwords. Auto-rotate keys (or use cert-based SSH like Teleport).
Tip	Keep an <code>~/.ssh/config</code> file with aliases for every server you touch. Saves typing the IP every time.

SSH LocalForward (port forwarding)

Plain English	Make a remote service show up as if it's on your local machine.
Technical	Tunnels TCP traffic from <code>localhost:<port></code> on your laptop through the SSH connection to <code><remote-host>:<port></code> on the server.
Example	<code>ssh -L 8080:localhost:8080 user@server</code> — accessing <code>localhost:8080</code> on laptop hits port 8080 on the server.
Why it matters	Access internal/firewalled services without exposing them publicly. We use this constantly in the lab.
Gotcha	The forwarded service might bind to 127.0.0.1 only on the server — that's fine, the tunnel target side is “localhost from the server's perspective”.

systemd

Plain English	The boss process that starts and watches all the services on a Linux system.
----------------------	--

Technical	An init system and service manager for Linux. PID 1 on most modern distros.
Why it matters	All your long-running daemons (nginx, postgres, docker) live under systemd. Diagnose with <code>systemctl status</code> and <code>journalctl</code> .
Gotcha	Containers don't need systemd — the container runtime IS the supervisor. Running systemd in a container is rare and awkward.

Section C — Git

Repository (repo)

Plain English	A project's folder with version history attached.
Technical	A directory containing a <code>.git/</code> subdirectory that holds the entire history of changes.
Tip	Always start projects with <code>git init</code> or <code>git clone</code> . Even for “just experimenting” — you'll thank yourself later.

Commit

Plain English	A snapshot of your project at a moment in time, with a message describing what changed.
Technical	A Git object containing a tree (file states), author/commmitter info, parent commit references, and a message. Identified by a 40-char SHA hash.
Why it matters	Smaller commits = easier to review, easier to revert, better blame info.
Tip	Write commit messages in imperative mood: “Add X”, not “Added X”. Industry convention.
Interview line	“We tag images by git short SHA. Every deploy is traceable to a specific commit — rollback is just <code>git revert</code> and ArgoCD reconciles.”

Branch

Plain English	A line of work, like a parallel timeline.
Technical	A movable pointer to a commit. The “current branch” is what HEAD points to.
Tip	Keep branches short-lived. Long branches = painful merges.

Gotcha	<code>git checkout</code> does two things (switch branch, restore file). Newer Git splits this into <code>git switch</code> and <code>git restore</code> .
--------	--

Pull Request (PR) / Merge Request (MR)

Plain English	“Please merge my branch into the main branch.” A GitHub/GitLab concept, not Git itself.
Technical	A web-UI workflow for proposing changes from one branch to another, with review, comments, CI integration.
Why it matters	This is THE gate between work and production. Code review, automated tests, security scans all run here.
Tip	Open the PR early as draft. Use the description for what + why + how-to-test. Reviewers love that.

Rebase vs Merge

Plain English	Two ways to combine branches. Merge keeps history; rebase rewrites it linear.
Technical	<code>merge</code> creates a new commit with two parents. <code>rebase</code> replays your commits on top of the target branch, creating new commits.
Why it matters	Rebased history is cleaner. Merge history shows what actually happened. Teams pick one.
Gotcha	Never rebase shared branches — you rewrite history that others have based on. Local branches: rebase. Shared branches: merge.
Tip	Use <code>git pull --rebase origin master</code> to keep your local work on top of the latest master.

Merge conflict

Plain English	Two changes on the same line — Git can’t decide. You decide.
Technical	When the 3-way merge algorithm can’t auto-resolve overlapping changes.
How to fix	Open the file. Find <<<<<< / ===== / >>>>>> markers. Pick what you want. Remove the markers. <code>git add</code> and continue.

Tip

```
git diff --name-only --diff-filter=U
```

lists files with conflicts.

Section D — Networking

IP Address

Plain English	A computer's mailing address on a network.
Technical	A 32-bit (IPv4) or 128-bit (IPv6) number identifying a network interface.
Why it matters	Everything network-related ultimately resolves to IP addresses.
Gotcha	A single host can have many IPs (one per network interface, multiple per interface).

CIDR (Classless Inter-Domain Routing)

Plain English	A compact way to write “this range of IPs”.
Technical	IP address followed by /N, where N is the count of network-prefix bits. /24 = first 24 bits fixed, last 8 bits free = 256 addresses.
Tip	Memorize: /8 = 16M, /16 = 64K, /24 = 256, /27 = 32, /30 = 4, /32 = 1.
Why it matters	Cloud VNets and subnets are sized in CIDR. Get this wrong and you can't fit your pods.

Subnet

Plain English	A slice of a larger network.
Technical	A range of IPs within a VNet, often with its own routing and security rules.
Why it matters	Logical separation — snet-aks for cluster, snet-mgmt for jumpboxes, snet-apps for future services.
Tip	Plan subnet sizes BEFORE deploying. Resizing is hard.

NSG (Network Security Group)

Plain English	A stateful firewall attached to subnets or VMs.
Technical	An Azure resource holding a list of allow/deny rules with priorities. Connection tracking means return traffic is auto-allowed.
Gotcha	High-priority Deny rules shadow lower-priority Allow rules. Including Azure's IMPLICIT Allow rules. Our lab broke iter 4 with this.
Tip	If you write an explicit "DenyAll" rule at priority 4000, you MUST add explicit <code>AllowVnetInbound</code> and <code>AllowAzureLoadBalancerInbound</code> at lower priorities.

DNS

Plain English	The phone book of the internet. Names to numbers.
Technical	Distributed hierarchical naming system. Resolvers query authoritative servers for records (A, AAAA, CNAME, MX, TXT).
In Kubernetes	CoreDNS resolves cluster-internal names like <code>backend.dev.svc.cluster.local</code> .
Gotcha	DNS caching delays propagation. Set TTLs intentionally.

Port

Plain English	A door number on a computer. Different apps listen on different ports.
Technical	A 16-bit number (0-65535) identifying a specific endpoint on an IP address for TCP/UDP.
Common ports	22 SSH, 80 HTTP, 443 HTTPS, 5432 Postgres, 6443 Kubernetes API.
Tip	Ports below 1024 require root to bind. Always use 1024 for user apps.

TLS / HTTPS

Plain English	Encrypted communication over the internet. The padlock in the browser.
----------------------	--

Technical	Transport Layer Security — protocol for confidentiality, integrity, and authentication using asymmetric (handshake) + symmetric (bulk) cryptography.
Why it matters	All public traffic should be HTTPS. Service-to-service inside clusters should use mTLS.
Gotcha	TLS certs expire. Auto-renew with cert-manager.

mTLS (mutual TLS)

Plain English	TLS where BOTH sides prove who they are.
Technical	Bidirectional certificate validation. Both client and server present certificates.
Why it matters	Service mesh (Istio) uses mTLS so services authenticate each other, not just users. Zero-trust networking.
Tip	Istio “PERMISSIVE” mTLS accepts both plaintext and mTLS during rollout. “STRICT” rejects plaintext.

Load Balancer

Plain English	Distributes incoming traffic across multiple backend servers.
Technical	Operates at L4 (TCP/UDP, like Azure LB) or L7 (HTTP, like Application Gateway or Istio ingress).
Why it matters	Single point of entry, automatic failover, traffic spreading.
Gotcha	L4 LBs don’t understand HTTP — they can’t route by URL or header.

Section E — Containers

Container

Plain English	A lightweight, isolated package containing an app and everything it needs to run.
Technical	A process or group of processes isolated by Linux kernel features (namespaces, cgroups, union FS).
Tip	A container is NOT a tiny VM. It shares the host kernel.
Why it matters	Containers solve “works on my machine” — same image runs identically anywhere.

Image

Plain English	A blueprint for a container.
Technical	An ordered set of read-only filesystem layers + metadata (config, entrypoint, env vars), identified by a digest (SHA).
Gotcha	Images can be huge. Aim for small base images (alpine, distroless, scratch).
Tip	Tag images with the git SHA. <code>:latest</code> is a footgun — it changes silently.

Dockerfile

Plain English	A recipe for building a container image.
Technical	A text file with instructions (<code>FROM</code> , <code>RUN</code> , <code>COPY</code> , <code>CMD</code> , etc.) executed by <code>docker build</code> .
Tip	Order instructions by frequency of change — base image first (rarely changes), code last (changes often). Maximizes layer cache.

Layer

Plain English	One step in an image build.
Technical	A filesystem diff produced by a Dockerfile instruction. Stacked via overlayFS to produce the container's view.
Why it matters	Cached layers don't rebuild. Smart layer ordering = fast builds.
Gotcha	Each RUN creates a layer. Chain commands with <code>&&</code> to reduce layers.

Multi-stage build

Plain English	Build in one container, run in another. Throw away build tools in the final image.
Technical	A Dockerfile with multiple <code>FROM ... AS <name></code> stages. <code>COPY --from=<stage></code> copies between them.
Why it matters	Final image is smaller, has fewer CVEs, fewer attack vectors.
Example	Build Node app in <code>node:20</code> , run final from <code>nginx:alpine</code> with just the built files.

Registry

Plain English	Where Docker images live online.
Technical	A server speaking the OCI distribution spec, hosting images at namespaced paths.
Common ones	Docker Hub, ACR (Azure), ECR (AWS), GCR/Artifact Registry (GCP), GHCR (GitHub).
Tip	Use a private registry for production. Docker Hub has rate limits and security concerns.

Image digest

Plain English	A cryptographic fingerprint of an image. Truly unique.
Technical	A SHA-256 hash of the image's manifest. Written as <code>myimage@sha256:abc....</code>
Why it matters	Tags can change ("latest" today vs tomorrow). Digests never change. Production-grade pinning.

Interview line

“We’re moving from tag-pinning to
digest-pinning so even a malicious registry push
can’t poison our deploys.”

Section F — Kubernetes

Pod

Plain English	One or more containers that always live together on the same node, sharing network and storage.
Technical	The smallest deployable unit in Kubernetes. A scheduling boundary.
Why it matters	Single container per Pod is the common case. Multi-container (“sidecar pattern”) is for logging agents, mesh proxies, init logic.
Gotcha	You almost never create Pods directly. Use Deployments, StatefulSets, Jobs.

Node

Plain English	A worker machine in the cluster. Runs Pods.
Technical	A VM (or physical) machine with kubelet (talks to API server), kube-proxy (networking), and a container runtime (containerd).
Tip	More smaller nodes = less complexity. Fewer larger nodes are often easier to manage but worse for bin-packing.

Namespace

Plain English	A folder inside the cluster.
Technical	A scope for names. Most objects (Pods, Services, ConfigMaps) are namespaced; some (Nodes, ClusterRoles) are not.
Why it matters	Logical separation between teams, environments, or apps. RBAC and quotas attach to namespaces.
Tip	Use namespaces aggressively. They’re free.

Deployment

Plain English	A way to run N copies of a Pod and update them safely.
Technical	A higher-level object that manages a ReplicaSet, which manages Pods. Provides rolling updates and rollbacks.
Why it matters	The default way to run stateless apps.
Gotcha	<code>replicas: 3</code> doesn't always mean 3 are running — readiness probes can keep some “not ready”.

ReplicaSet

Plain English	“Make sure exactly N copies of this Pod exist.”
Technical	A controller that ensures the desired number of Pod replicas matching a selector. Usually managed by a Deployment.
Tip	You rarely manage ReplicaSets directly. Look at them when debugging weird Deployment behavior.

StatefulSet

Plain English	Like Deployment but for things with identity — databases, brokers, leader-elected systems.
Technical	Manages Pods with stable names (<code>db-0</code> , <code>db-1</code>), stable network identity, stable storage (PVCs per ordinal), ordered start/stop.
Gotcha	Scaling down a StatefulSet doesn't delete the PVCs. Data sticks around until you delete them explicitly.

Service

Plain English	A stable network endpoint for a set of Pods. Pods come and go; Service stays.
Technical	Object with a selector and ports. kube-proxy programs iptables/IPVS to load-balance to backing Pods.
Types	<code>ClusterIP</code> (internal), <code>NodePort</code> (each node's IP), <code>LoadBalancer</code> (cloud LB), <code>ExternalName</code> (DNS alias).

Tip	Default ClusterIP is enough for most cases. Use Ingress/Gateway for external traffic.
------------	---

ConfigMap

Plain English	A bag of key-value config for your app.
Technical	A namespaced object holding string data. Mountable as files or env vars.
Why it matters	Decouple config from code/images. Same image, different ConfigMap per environment.
Gotcha	ConfigMap data isn't encrypted. Don't use for secrets.

Secret

Plain English	A bag of sensitive config for your app.
Technical	Like ConfigMap but base64-encoded. Optionally encrypted at rest (etcd).
Gotcha	“Base64-encoded” is NOT encryption. Real production uses External Secrets + KMS.
Tip	Enable etcd encryption-at-rest on your cluster. Easy to forget.

PersistentVolume / PersistentVolumeClaim

Plain English	Persistent storage that survives Pod restarts.
Technical	PV is the actual storage resource (provisioned by StorageClass). PVC is a request for storage that gets bound to a PV.
Why it matters	Without PV/PVC, restarting a Pod loses everything in its filesystem.
Gotcha	<code>ReadWriteOnce</code> access mode means one node at a time. Multiple Pods on different nodes can't share. Use <code>ReadWriteMany</code> if needed (Azure Files, NFS).

Ingress / Gateway

Plain English	The “front door” of the cluster — public URL routing.
----------------------	---

Technical	Ingress = older API. Gateway API = modern replacement. Both define how external traffic reaches Services.
Why it matters	Without Ingress/Gateway, every Service needs its own LoadBalancer (expensive) or NodePort (ugly).
In our lab	Istio Gateway + VirtualService is our front door. Routes by Host header.

ServiceAccount

Plain English	A workload's identity. "Who is this Pod?"
Technical	A namespaced object representing a non-human identity. Bound to RBAC roles.
Why it matters	Every Pod has a ServiceAccount (default default if you don't specify). Used for talking to the Kubernetes API.
Tip	Never use default . Create a specific SA per workload.

RBAC (Role-Based Access Control)

Plain English	Who can do what in the cluster.
Technical	Roles (sets of permissions) bound to subjects (users, groups, ServiceAccounts) via RoleBindings.
Tip	Least privilege. Start with no permissions, add only what's needed. Audit quarterly.
Gotcha	ClusterRole + ClusterRoleBinding is cluster-wide. Easy to over-grant. Prefer Role + RoleBinding (namespace-scoped).

Controller

Plain English	A program that watches the cluster and makes things match the desired state.
Technical	A control loop that reconciles actual state → desired state . Built-in (DeploymentController) or custom (Operators).
Why it matters	The "Kubernetes way" — declarative, eventually consistent. You declare what you want, controllers make it so.

Interview line	“Argo Rollouts is a custom controller — it has its own CRD (Rollout) and reconciles by managing ReplicaSets and Services itself, just like the built-in DeploymentController.”
-----------------------	--

CRD (Custom Resource Definition)

Plain English	A way to define your own Kubernetes object types.
Technical	A YAML manifest that adds a new API kind to the cluster. Coupled with a controller that knows what to do with it.
Why it matters	Argo Rollouts, ArgoCD, Istio, Prometheus all add CRDs. The pattern is everywhere.

Operator

Plain English	A custom controller that knows about a specific application.
Technical	CRD(s) + controller that encodes operational expertise (install, upgrade, backup, scale) for a specific app.
Examples	Prometheus Operator, Cert Manager, ArgoCD, KEDA.
Tip	If you find yourself writing complex Helm charts to handle “if upgrading from version X, run job Y” — that’s a sign you want an Operator.

Section G — Terraform

Infrastructure as Code (IaC)

Plain English	Describe infrastructure in text files instead of clicking in cloud consoles.
Why it matters	Reproducible, reviewable, versioned. Same code anywhere. Destroy + rebuild from scratch.
Tools	Terraform (multi-cloud), Pulumi (real languages), CloudFormation (AWS), ARM/Bicep (Azure native).
Interview line	“We use Terraform because it’s cloud-agnostic, has the largest provider ecosystem, and a clear declarative model.”

State (Terraform state)

Plain English	A file mapping your code to real cloud resources.
Technical	A JSON file (<code>terraform.tfstate</code>) tracking resource IDs, dependencies, and attributes.
Why it matters	Without state, Terraform can’t compute diffs. State IS the source of truth for “what exists”.
Gotcha	State contains secrets (passwords, keys). Encrypt at rest. Never commit to Git.

Remote backend

Plain English	Store state in the cloud, not on your laptop.
Technical	A state storage location (Azure Storage, S3, etc.) supporting locking for concurrent applies.
Why it matters	Team collaboration. Without it, two people running apply simultaneously corrupts state.
Tip	Always use remote backend in production. Local state is for personal toys only.

Provider

Plain English	A plugin Terraform uses to talk to a specific cloud or service.
Technical	A Go binary that translates Terraform HCL operations into API calls. <code>hashicorp/azurerm</code> , <code>hashicorp/kubernetes</code> , etc.
Tip	Pin provider versions (<code>version = "~> 3.0"</code>) to avoid surprise breakage.

Module

Plain English	A reusable bundle of Terraform code.
Technical	A directory of <code>.tf</code> files invocable with <code>module "name" { source = "..."}.</code>
Why it matters	DRY. Define an “aks-cluster” module once, use it in dev/staging/prod.
Tip	Publish modules to a private registry (Terraform Cloud) or pin them by Git tag.

State locking

Plain English	“Don’t let two people change state at once.”
Technical	Backend acquires a lock (blob lease in Azure, DynamoDB row in AWS) before apply. Releases on completion.
Why it matters	Prevents state corruption from concurrent writes.
Gotcha	If <code>terraform apply</code> crashes, locks can stick. Use <code>terraform force-unlock</code> carefully.

Plan vs Apply

Plain English	Plan = “show me what would change”. Apply = “do it”.
Tip	Always <code>plan</code> first. Save with <code>-out=tfplan</code> , then <code>apply tfplan</code> — guarantees no surprise changes between plan and apply.
Why it matters	Every cloud change should be previewed before execution.

Drift

Plain English	Difference between what Terraform thinks exists and what really exists.
Technical	When real cloud resources are modified outside Terraform (manual edits, other tools), state goes out of sync.
Fix	<code>terraform refresh</code> (or <code>terraform plan</code> which refreshes implicitly) shows the drift. Adopt the changes or revert.

Section H — Helm

Helm

Plain English	A package manager for Kubernetes.
Technical	Templates + values combine to produce manifests. Track installed packages (Releases).
Why it matters	Don't reinvent install/upgrade logic for every app. Reuse community charts.
Tip	Use Helm for installs of third-party software. Hand-write or kustomize for in-house apps.

Chart

Plain English	A package of Kubernetes manifests with parameters.
Technical	Directory with <code>Chart.yaml</code> , <code>values.yaml</code> , and <code>templates/</code> containing Go templated YAML.

Release

Plain English	An installation of a chart with a specific name.
Technical	A named, versioned deployment of a chart in a namespace. Each <code>helm install</code> creates a Release.
Tip	Use <code>helm upgrade --install</code> for idempotency — installs if missing, upgrades if exists.

Values

Plain English	Variables you pass to a chart.
Technical	YAML data merged with chart defaults to produce the final manifests.

Tip	Separate values per environment: <code>values-dev.yaml</code> , <code>values-prod.yaml</code> .
------------	--

Umbrella chart

Plain English	A chart that depends on other charts.
Technical	A parent chart with <code>dependencies:</code> in <code>Chart.yaml</code> . Sub-charts under <code>charts/</code> .
Why it matters	Compose complex apps (frontend + backend + DB) into one Helm release.
In lab	Our three-tier chart has subcharts for frontend, backend, database.

Section I — Azure

Service Principal (SP)

Plain English	A non-human identity for automation tools.
Technical	An Azure AD application with a client ID and a secret (or cert). Has RBAC role assignments.
Why it matters	CI tools and scripts need an identity. SP is the explicit one.
Gotcha	SPs have long-lived secrets. Rotate them. Or use Workload Identity instead (no secrets).
Interview line	“We use SP for the Terraform pipeline because we run it outside Azure. Inside Azure (AKS), we’d use Workload Identity to eliminate long-lived secrets.”

Managed Identity

Plain English	An automatically managed identity for Azure resources.
Technical	Azure resource gets an identity, Azure handles credential rotation. Two types: System-Assigned (tied to resource lifecycle) and User-Assigned (independent).
Why it matters	No secrets to store, no rotation to manage. The cloud handles it.
Tip	Always prefer Managed Identity over Service Principal when possible.

Workload Identity (AKS)

Plain English	Pods get Azure AD identities without storing secrets.
----------------------	---

Technical	OIDC federation — Azure trusts tokens issued by your AKS cluster’s OIDC issuer. Pods get a JWT, exchange it for an Azure AD access token.
Why it matters	Modern, secure, no long-lived credentials.
Gotcha	Requires <code>--enable-oidc-issuer</code> and <code>--enable-workload-identity</code> flags on AKS. Older clusters need an upgrade.

AKS (Azure Kubernetes Service)

Plain English	Managed Kubernetes on Azure.
Technical	Azure runs the control plane (free in non-paid SKU). You manage worker nodes.
Tip	<code>az aks stop</code> saves money — control plane stays free, nodes don’t bill.

ACR (Azure Container Registry)

Plain English	Docker image hosting on Azure.
Technical	OCI-compliant registry. Three SKUs: Basic, Standard, Premium (with geo-replication).
Tip	<code>az aks update --attach-acr <acr></code> wires permissions so cluster can pull images. No pull secrets needed.

Log Analytics Workspace (LAW)

Plain English	Centralized log storage.
Technical	Azure Monitor’s underlying log storage. Queried with Kusto Query Language (KQL).
Why it matters	AKS sends container logs here via OMS agent. App logs, audit logs, system events all land here.
Gotcha	Costs scale with ingestion volume. Set retention carefully.

Section J — CI/CD

CI (Continuous Integration)

Plain English	Every code change is automatically built and tested.
Technical	Pipeline that runs on every commit/PR — compile, lint, test, package.
Why it matters	Catch bugs early. Enforce standards. Confidence to merge.

CD (Continuous Delivery vs Deployment)

Plain English	Delivery = always ready to deploy. Deployment = auto-deploy on every successful CI.
Why it matters	The difference is whether a human pushes the deploy button.
Tip	Most teams use “CD” loosely. Be precise in interviews — “we have continuous delivery; production deploys are gated by an approver”.

Pipeline

Plain English	An automated sequence of build/test/deploy steps.
Technical	Code (Jenkinsfile, ci-cd.yaml) defining stages, jobs, and conditions.
Tip	Pipelines should be in the repo (pipelines as code), not in the CI tool’s UI.

Artifact

Plain English	The output of a build that you keep — image, JAR, ZIP.
Technical	A built, immutable output of CI, stored for traceability and reuse.
Tip	Tag artifacts with git SHA. Promote the same artifact through environments — don't rebuild for prod.

GitOps

Plain English	Git is the source of truth for what should be deployed.
Technical	A controller (ArgoCD, Flux) watches Git and applies changes to the cluster.
Why it matters	Audit trail, easy rollback (git revert), declarative.
Interview line	“We adopted GitOps so every deploy is a Git commit reviewable in <code>git log</code> . Rollback = <code>git revert</code> . No more ‘cluster diverged from Git silently’.”

Image promotion (build-once-deploy-many)

Plain English	Same image goes through dev → staging → prod. Don't rebuild per environment.
Technical	CI builds one image with a unique tag (git SHA), promotes by updating values.yaml in each env.
Why it matters	The image you tested in dev is byte-for-byte the image in prod. Eliminates “works in staging” mystery.

Section K — Progressive Delivery

Blue/Green deployment

Plain English	Run two versions side-by-side. Switch traffic over at once.
Technical	Active ReplicaSet (blue) serves traffic. New version (green) deployed alongside. Service selector flips from blue to green when ready.
Why it matters	Instant rollback — keep blue running for X seconds after promotion.
Gotcha	Costs 2x resources during deploy.

Canary deployment

Plain English	Gradually shift X% of traffic to the new version. Watch. Increase.
Technical	Route a percentage of traffic to new ReplicaSet via Service mesh or Ingress weights. Increase based on health.
Why it matters	Catch issues that only manifest at scale, with real users, on a small slice.
Tip	Pair with metric-based analysis. If error rate spikes during the 10% phase, auto-abort.

Argo Rollouts

Plain English	Kubernetes controller that does blue/green and canary deployments.
Technical	CRD (Rollout) replacing Deployment. Manages ReplicaSets, Services, analysis.
Why it matters	First-class progressive delivery, not bolt-on.
Commands	<code>kubectl argo rollouts promote, abort, undo, retry rollout.</code>

AnalysisTemplate

Plain English	“Check metrics before promoting.”
Technical	Argo Rollouts CRD defining queries (against Prometheus, Datadog, etc.) and success/failure conditions.
Why it matters	Metric-gated promotion = automatic safety net.
Interview line	“Our AnalysisTemplate has two gates: min-traffic (1 req/sec on preview) and success-rate (95% non-5xx). Both must pass 3 consecutive checks before promotion.”

Min-traffic gate

Plain English	“Don’t trust the success-rate gate if no traffic is flowing.”
Technical	Pre-promotion analysis metric that requires a minimum request rate.
Why it matters	Without it, <code>or vector(1)</code> falsely returns 100% success on zero traffic. Min-traffic gate forces real traffic to flow before evaluation.

Section L — Observability

Metric

Plain English	A number measured over time.
Technical	Time-series data point: name, labels, value, timestamp.
Example	<code>http_requests_total{method="GET",status="200"} 1234</code>

Cardinality

Plain English	How many unique combinations of labels a metric has.
Technical	Each unique label tuple = one time series. Sum across all metrics = cardinality.
Gotcha	High cardinality kills Prometheus (memory). Don't use unbounded values as labels (<code>user_id</code> , <code>request_id</code>).
Tip	Use histogram buckets, not unique values, when you can.

PromQL

Plain English	The query language for Prometheus.
Technical	Functional, declarative language for selecting and aggregating time-series.
Example	<code>sum by (status) (rate(http_requests_total[5m]))</code> — request rate grouped by status, over 5-minute window.

SLI / SLO / SLA

SLI	Service Level Indicator — a metric (e.g., success rate).
SLO	Service Level Objective — an internal target (e.g., 99.9%).
SLA	Service Level Agreement — a contract with consequences.
Tip	Define SLOs from the user's perspective, not from CPU/memory.

Error budget

Plain English	How much downtime you're allowed before missing your SLO.
Technical	$(1 - \text{SLO}) \times \text{time}$. For 99.9% over 30 days, error budget = 43 minutes.
Why it matters	Burn rate alerting — page on-call when budget is being consumed too fast.

Reading the Handbook — Pro Tips

1. **Don't read straight through.** Read one chapter, do the exercises, come back next day.
 2. **Type, don't copy.** Every command, every YAML — type it. Muscle memory matters.
 3. **Break things on purpose.** Best learning is recovery from mistakes you made.
 4. **Use the labs you've built.** This handbook is paired with a real working repo. Run it.
 5. **Talk through concepts.** Explain to a rubber duck. If you can't explain it, you don't understand it.
 6. **Take notes in your own words.** Don't highlight the handbook. Write your own summary.
 7. **Drill the interview questions.** Every chapter has them — actually rehearse out loud.
 8. **Compare to your previous knowledge.** “How is Kubernetes Service like a load balancer I already know?”
-

Common Cross-Cutting Tips

When you hit an error

1. **Read the error.** Slowly. Twice. 80% of errors tell you exactly what's wrong.
2. **Look at events.** `kubectl get events -A --sort-by='.lastTimestamp' | tail` shows recent issues.
3. **Check logs.** `kubectl logs <pod>` for app, `journalctl -u <service>` for systemd.
4. **Search the exact error message.** Verbatim, in quotes, in Google. Someone hit it before.
5. **Bisect.** Did it work yesterday? `git log` the last day's changes. Revert one at a time.

When learning a new tool

1. **Read the quickstart.** Just the quickstart. Resist the urge to read the whole docs.
2. **Build something small.** A Hello World, then your real use case.
3. **Read the source/issues.** Especially GitHub Issues — that's where real problems are documented.
4. **Find the mental model.** Every tool has one. ArgoCD = “sync Git to cluster”. Terraform = “diff state to cloud”.

When debugging Kubernetes

1. `kubectl get pods -A` - anything not Running?
2. `kubectl describe pod <name>` - events at the bottom?
3. `kubectl logs <name>` - what did the app say?
4. `kubectl logs <name> --previous` - if it crashed, last instance's logs
5. `kubectl exec -it <name> -- sh` - get a shell, poke around
6. `kubectl get events --sort-by='.lastTimestamp' | tail`
7. Try the same in a different namespace - namespace issue?
8. Try the same with a different image - image issue?
9. Try the same on a different cluster - cluster issue?

When writing code/YAML/HCL

1. **Format on save.** Use editor plugins. Manual formatting wastes time.
2. **Validate before commit.** `terraform validate`, `helm lint`, `kubectl apply --dry-run=client`.
3. **Commit small.** One logical change per commit.

4. **Write the test first** (or at least the test plan in the PR description).

Career tips

1. **Document everything.** Future you will thank you. Your team will too.
 2. **Automate the boring stuff.** If you do it 3 times, script it. 5 times, productize it.
 3. **Share your work.** Blog posts, talks, internal wikis. Visibility = career growth.
 4. **Learn one new thing every week.** Even 30 minutes of focused study compounds.
 5. **Be honest about gaps.** “I don’t know X, but here’s how I’d learn it” is a great answer.
-

End of Glossary